

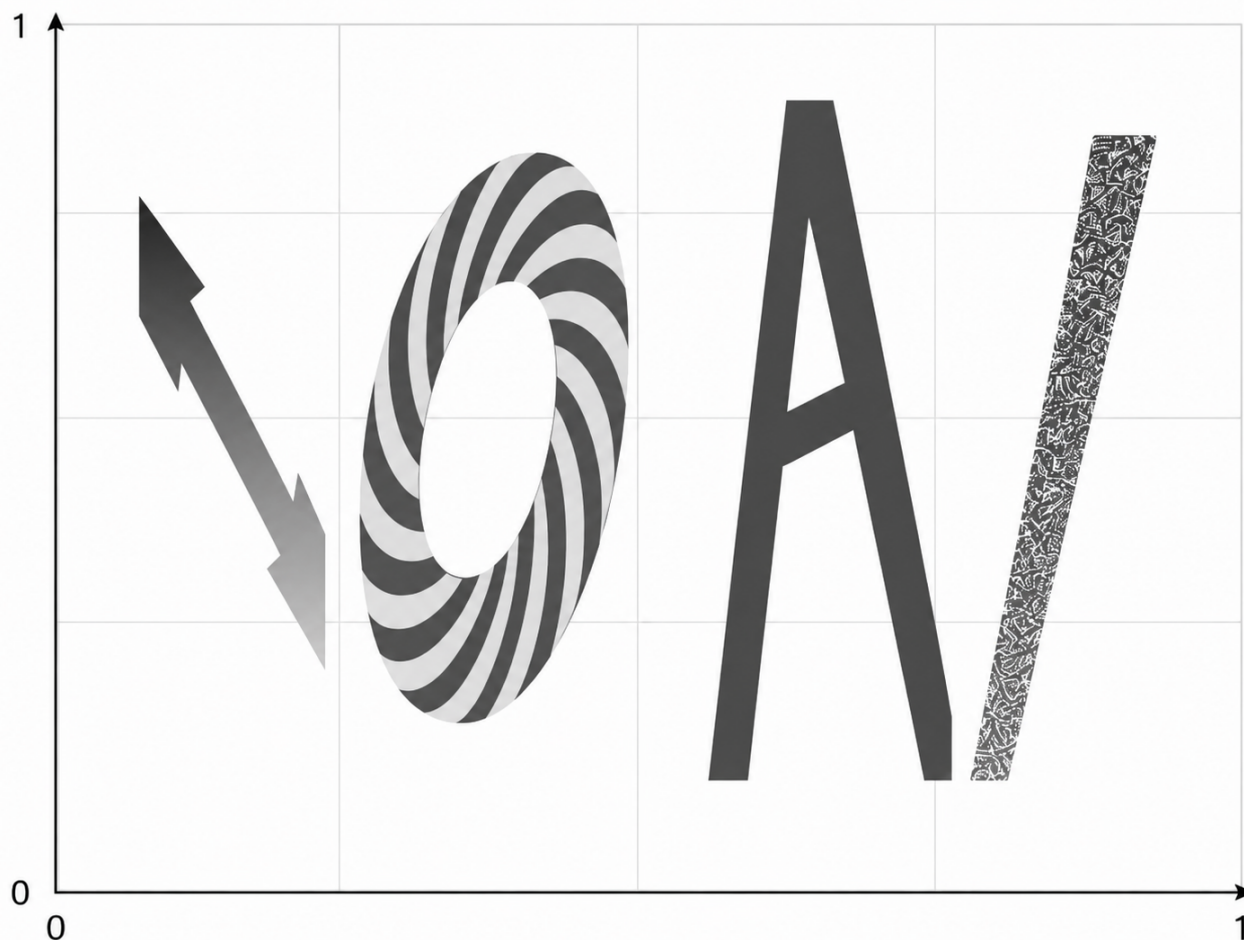
## IOAI поле

- **Ограничение за време:** 5 минути
- **Дисково пространство:** 5 GB
- **Размер на решението:** `solution.ipynb`, `custom_model.py`  $\leq 1$  MB общо
- **Предварително обучени модели:** няма — обучавайте от нулата, без интернет по време на оценяването
- **Базов резултат:** 31.2187

## Задача

Кметът на Астана иска да украси града със стилизирани логотипи на IOAI. Като статистик той разглежда всичко — включително логотипа — като пространствена функция  $F(x, y, \overline{W})$ , където  $x, y \in [0, 1]$  представляват координати в 2D равнина, а  $\overline{W}$  е множество от скрити параметри, определящи стилистични характеристики като цветовете и ъглите на буквите.

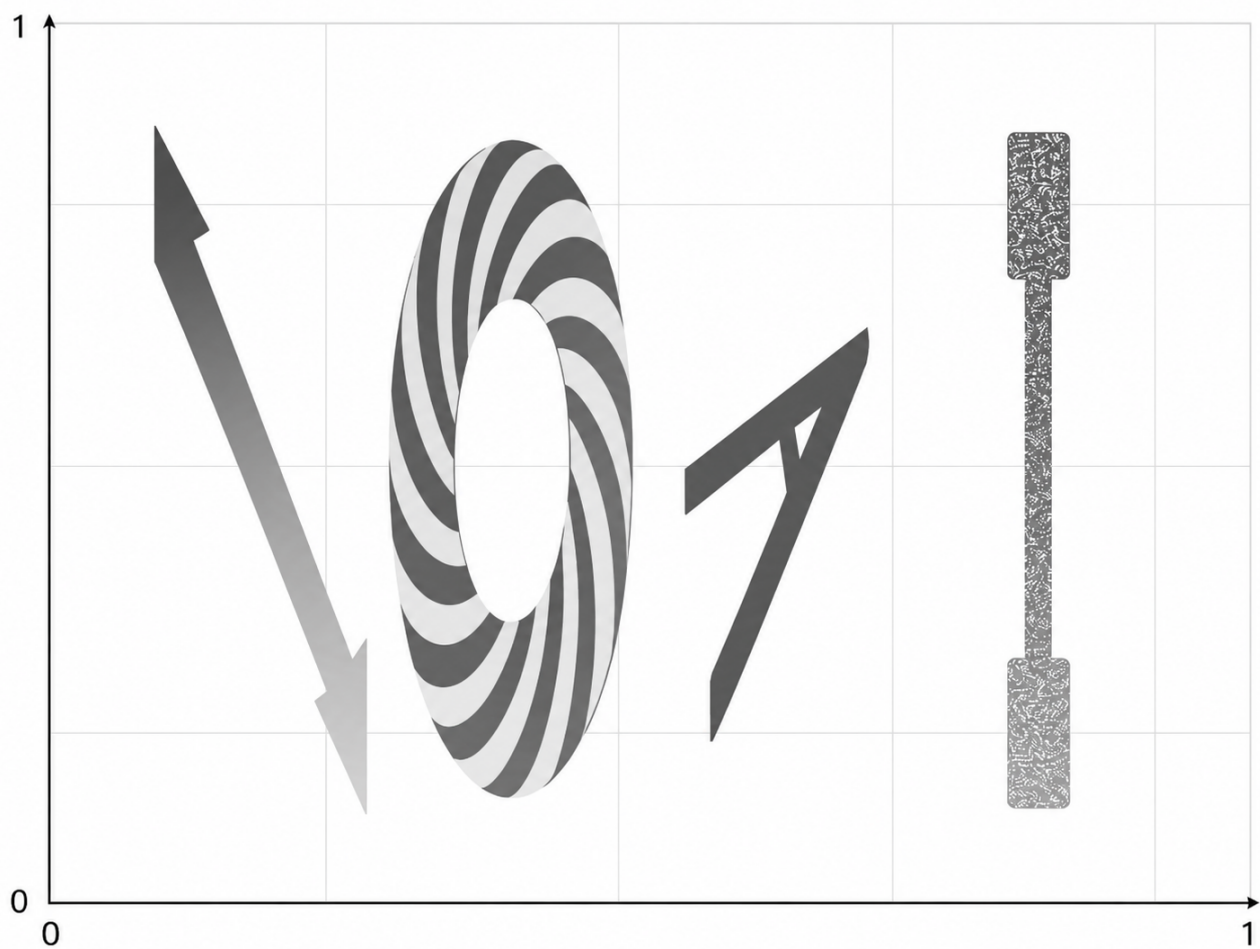
Тъй като  $F$  е твърде сложна, за да бъде изразена като явно математическо уравнение, Вашата задача е да обучите невронна мрежа, която да я апроксимира. Мрежата ще извежда стойност на **IOAI полето** за всяка двойка координати  $(x, y)$ , като генерира пълна визуализация на логотипа чрез топлинна карта в равнината. Ето пример за визуализация чрез топлинна карта на  $F$  с конкретни скрити параметри  $\overline{W}$ .

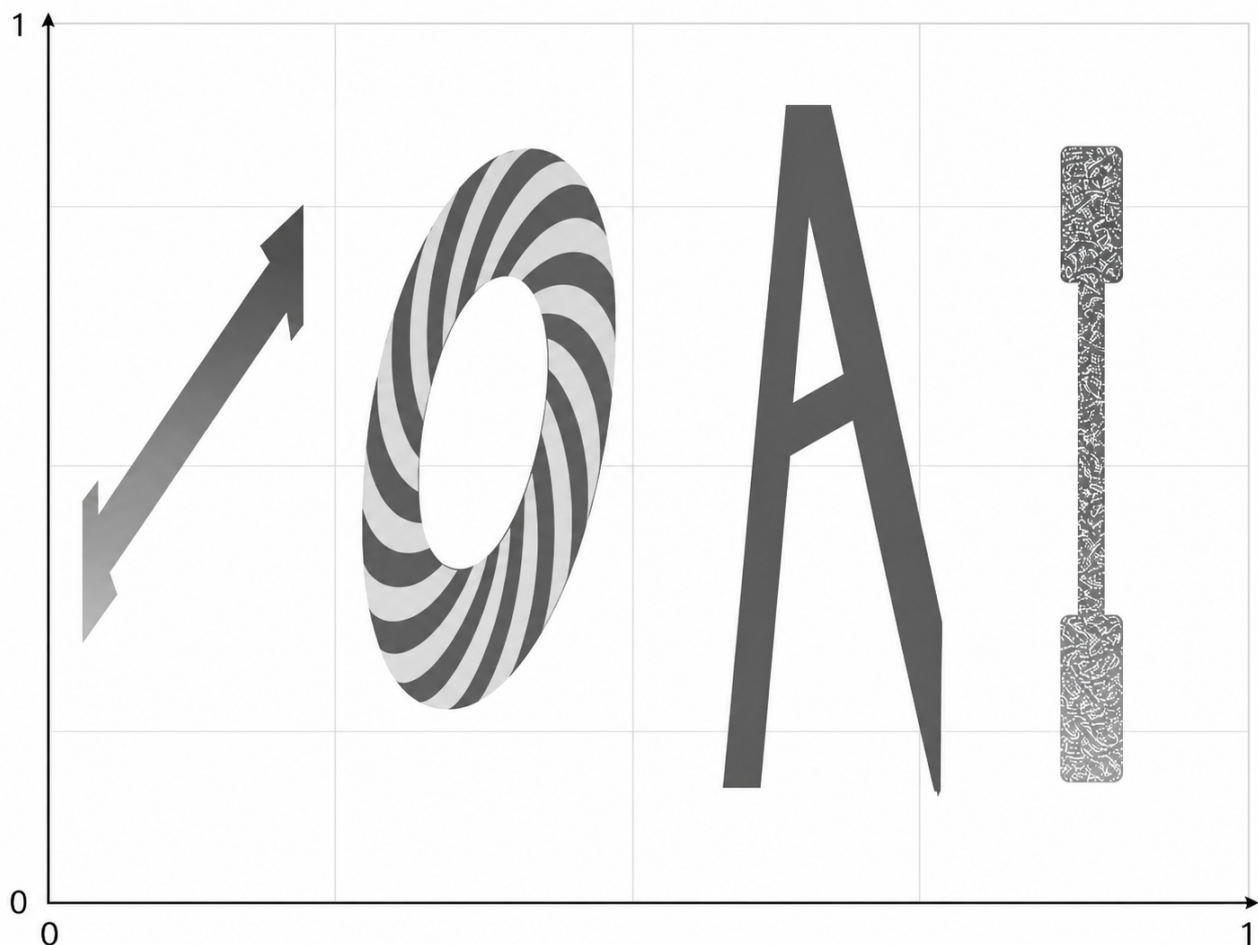


От какво се състои IOAI полето? От четири букви и фон.

- Стойностите в първата буква **I** са много големи ( $1e+10$  и повече) и имат линеен градиент
- Стойностите в буквата **O** образуват спираловиден шаблон
- Стойността в буквата **A** винаги е  $-1$
- Стойностите в последната буква **I** трябва да бъдат случайни стойности от диапазона  $[-2026, 2026]$  дори ако бъдат изчислени два пъти в една и съща точка
- Извън буквите стойността винаги е нула

Функцията има скрити параметри  $\overline{W}$ , които влияят върху мащаба и наклона на буквите, както и върху диапазона от стойности в първата буква **I**. Буквите обаче няма да се пресичат. Ето няколко илюстративни примера за това как изглежда IOAI полето при различни  $\overline{W}$ :





### Какво Ви е предоставено:

Тази задача НЕ съдържа набори от данни. Вместо това Ви е предоставена функцията генератор, която се конфигурира чрез JSON конфигурационния файл на `data/train_config/field_config.json`.

Тестовата конфигурация е скрита, но е от подобно естество. Вашата задача е да обучите модел върху предоставения генератор, като използвате толкова данни, колкото желаете. Вашите "train" и "test" разпределения се генерират от един и същ генератор — просто не знаете в кои точки  $(x_i, y_i)$  ще бъдете оценявани.

Решението Ви трябва да се състои от:

- клас на модела за обучение, записан като `custom_model.py`. Този модел трябва да наследява класа `torch.nn.Module` и да използва само импорти от `torch`. Той трябва да съдържа класа `CustomModel`, използван в тетрадката `solution.ipynb`.
- тетрадка `solution.ipynb`, която ще създаде теглата `model.pt`

## Оценяване

За всяка област минималният резултат е 0, а максималният резултат е 1. Крайният резултат е средната стойност за всичките пет области (четири за всяка буква и една за фона), умножена по 100. Има **наказание за броя параметри**:

**Ако моделът Ви има повече от 20260 параметъра, резултатът се намалява наполовина.**

Броят на параметрите се измерва чрез `sum(p.numel() for p in model.parameters())`. Очакваме моделът Ви да работи и в стохастичен режим, като PyTorch `nn.Dropout` е част от модела.

### За стандартните области

За всяка област  $R$  (първата буква I, O, A, Background) оценяваме модела върху  $N_R = 512$  тестови точки  $(x_i, y_i)$  с истински стойности  $v_i$  и предсказания  $\hat{v}_i$ . Като основна метрика използваме нормализирана средна абсолютна грешка (MAE). MAE се дефинира като:

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

А нормализацията се извършва по следния начин:

$$\text{score}(R) = 1 - \min \left( \frac{\text{MAE}(R)}{s_R}, 1 \right),$$

където  $s_R > 0$  е константа за мащабиране.

### За областта на последната буква I

В тази област **dropout е активиран по време на оценяването**. За всяка тестова точка  $j$ :

1. Изпълняваме модела  $K = 10$  пъти, за да получим  $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$ .
2. Ако някоя изходна стойност е извън диапазона  $[-2026, 2026]$ , тогава  $\text{pointScore}(j) = 0$ .
3. В противен случай изчисляваме стандартното отклонение  $\sigma_j$  на  $K$ -те изходни стойности и го преобразуваме в резултат:

$$\text{pointScore}(j) = \min \left( \frac{\sigma_j}{s_E}, 1 \right),$$

където  $s_E > 0$  е фиксирана константа за мащабиране.

Резултатът за областта е средната стойност за всички точки в нея:

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

където  $N_E = K * N_R$ .

Казано по-просто, колкото по-голямо разнообразие имате, толкова по-висок ще бъде резултатът Ви за тази област. **Не можете да използвате случайност в чист вид, включително функциите на PyTorch `rand*` и `_uniform`; случайността трябва да идва от инференцията с активиран `dropout`.**

## Как да предадете решението си

1. Отворете `solution.ipynb` и изпълнете всички клетки.
2. Подобрете модела `CustomModel` в `custom_model.py`
3. Уверете се, че последната Ви клетка записва модела Ви във файла `model.pt`.
4. В `Git` раздела на `JupyterLab` добавете към `staging`, въведете коментар и направете `commit` на `solution.ipynb` и `custom_model.py`, след което го изпратете.
5. Върнете се на страницата на състезанието и натиснете **Предаване**. Коментарът при предаване трябва да бъде същият като коментара от предходната стъпка.